

Devoir surveillé (Session Normale)

Remarque Générales :

- Toutes les copies doivent comporter votre nom complet ;
- Vous êtes invités à soigner la présentation de votre copie, à mettre en évidence les principaux résultats, à respecter les notations de l'énoncé
- Ce devoir est composé de deux parties indépendantes ;
- Toutes les instructions et les fonctions demandées seront écrites en **C** ;
- Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- Tous les appareils électroniques sont interdits ;

Partie 1 : Fichier informatique

Un fichier informatique est une collection d'informations numériques réunies sous un même nom, enregistrées sur un support de stockage tel qu'un disque dur, un CD-ROM ou une bande magnétique, et manipulées comme une unité.

Un fichier est représenté par la structure (enregistrement) suivante :

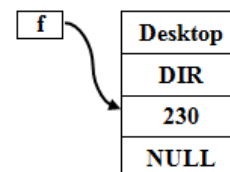
```
typedef struct fichier{  
    char nom[30];  
    char type[30];  
    int taille;  
    struct fichier *suiv;  
} Fichier;
```

avec **nom** : le nom de fichier , **type** : le type de fichier (image,texte,...) , **taille** : la taille en nombre d'octets du fichier et **suiv** : pointeur sur le suivant.

1. Écrire une fonction dans le prototype **Fichier *CreerFichier()** qui permet de créer, de saisir les informations d'un nouveau fichier fourni par clavier et de retourner l'adresse du fichier créé.

Exemple d'exécution : de **Fichier *f = Creer_Fichier()**

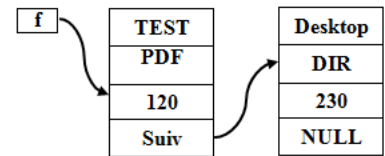
Entrer le nom de fichier maximum 30 caractère : Desktop Entrer le type de fichier maximum 30 caractère : DIR Entrer la taille de fichier : 230



2. Écrire une fonction dans le prototype **Fichier *Ajouter_Debut(Fichier *f)** qui reçoit une liste chaînée **f** et qui ajout un **nouveau** fichier au **début** de la liste **f**.

Exemple d'exécution : de **f = Ajoute_Debut(f)**

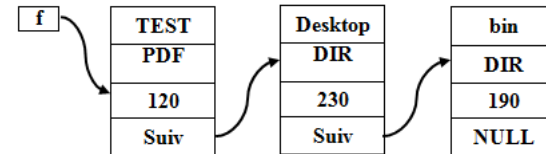
Entrer le nom de fichier maximum 30 caractère :
TEST
 Entrer le type de fichier maximum 30 caractère :
PDF
 Entrer la taille de fichier :
120



3. Écrire une fonction dans le prototype **Fichier *Ajouter_Fin(Fichier *f)** qui reçoit une liste chaînée **f** et qui ajout un **nouveau** fichier à la **fin** de la liste **f**.

Exemple d'exécution : de **f = Ajoute_Fin(f)**

Entrer le nom de fichier maximum 30 caractère :
bin
 Entrer le type de fichier maximum 30 caractère :
DIR
 Entrer la taille de fichier :
190



4. Écrire une fonction dans le prototype **int Rechercher_Nom(Fichier *f, nomFichier)** qui reçoit une liste chaînée **f** et un nom de fichier **nomFichier** et retourne **1** si le fichier existe dans la liste **f**, sinon la fonction retourne la valeur **0**.

Remarque : on peut utiliser la fonction **strcmp(str1,str2)** de la bibliothèque **<String.h>**

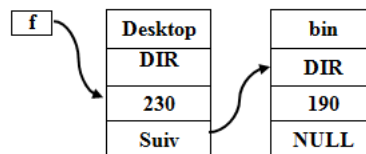
Exemple d'exécution :

l'appel de la fonction **Recherche_Nom(f,"Desktop")** retourne la valeur **1**

5. Écrire une fonction dans le prototype **Fichier *Supprimer_Debut(Fichier *f)** qui reçoit une liste chaînée **f** et qui permet de **supprimer** le **premier** fichier de la liste **f**.

Exemple d'exécution :de **f = Supprimer_Debut(f)**

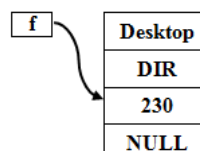
Le fichier dont le nom "**Test**" sera supprimer



6. Écrire une fonction dans le prototype **Fichier *Supprimer_Fin(Fichier *f)** qui reçoit une liste chaînée **f** et qui permet de **supprimer** le **dernier** fichier de la liste **f**.

Exemple d'exécution :de **f = Supprimer_Fin(f)**

Le fichier dont le nom "**bin**" sera supprimer

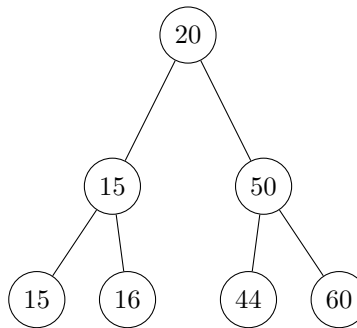


Partie 2 : Les arbres binaires de recherche

Un arbre binaire de recherche soit un arbre binaire vide (**NULL**) ou bien un triplet (**r,G,D**) (tel que **r** est la **racine** de l'arbre, **G** le **sous arbre gauche** de l'arbre et **D** est le **sous arbre droit** de l'arbre.) dont les conditions suivantes sont vérifiées :

- L'étiquette **r** de la racine de l'arbre binaire est supérieure ou égale à toutes les étiquettes du sous-arbre gauche **G** ;
- L'étiquette **r** de la racine de l'arbre binaire est inférieure strictement à toutes les étiquettes du sous-arbre droit **D** ;
- Les sous-arbres **G** et **D** sont des arbres binaires de recherche ;

Exemple :



1. Représenter graphiquement les arbres définis de la façon suivante :
 - **A** = (18,(16,(15,NULL,NULL),(16,NULL,NULL)),(22,NULL,(24,NULL,NULL)))
 - **B** = (24,NULL,(25,NULL,(26,NULL,(27,NULL,NULL))))
 - **C** = (34,(29,(24,NULL,NULL),(32,(31,NULL,NULL),NULL)),(37,(35,NULL,NULL),(40,NULL,NULL)))
2. Ces arbres binaires sont-ils de recherche ? sinon Justifié votre réponse.

On peut représenter un arbre binaire de recherche avec la structure suivante :

```
typedef struct arbre_binaire_recherche{
    int racine;
    struct arbre *SAG;
    struct arbre *SAD;
}ABR;
```

tel que le champs **racine** représente la racine de l'arbre, le champs ***SAG** représente le sous arbre gauche et ***SAD** représente le sous arbre droit.

3. Écrire une fonction **Creer_ABR(int valeur, ABR *ABG, ABR *ABD)** qui prend en argument une valeur entière ainsi que deux arbres binaire de recherche ABG et ABD et renvoie un arbre dont la racine contient cette valeur et les deux sous-arbres sont ceux donnés en paramètre.
4. Compléter le code suivant pour construire l'arbre C défini en Question 1.

```
int main() {
    ABR *C = Creer_ABR(34,....);
    return 0;
}
```

5. Écrire une fonction **Affiche_arbre(ABR *a)** permettant d'afficher les valeurs des noeuds d'un arbre binaire de recherche de manière à lire la structure de l'arbre. Un noeud sera affiché sous la forme **{G,r,D}** où **G** est le sous-arbre gauche, **r** la valeur du noeud et **D** le sous-arbre droit.
Par Exemple : L'arbre C sera affiché par : **{{{_,24,_},29,{{{_,31,_},32,_}},34,{{{_,35,_},37,{{_,40,_}}}}**.
où **'_'** représente l'arbre vide.

6. Écrire une fonction **parcours_infixe**(**ABR *a**) qui reçoit un arbre binaire de recherche et affiche le parcours infixé de l'arbre **a**.

parcours_infixe(C) : affiche **24-29-31-32-34-35-37-40**.

7. Écrire une fonction **recherche**(**ABR *a, int x**) qui reçoient en paramètres un arbre binaire de recherche **a** et une valeur entière **x** et renvoie **1** si la valeur **x** présent dans l'ABR **a** et **0** sinon.

recherche(C,28) : revoie **0**.

recherche(C,31) : revoie **1**.

8. Écrire une fonction **insérer**(**ABR *a, int x**) qui reçoient en paramètres un arbre binaire de recherche **a** et une valeur entière **x** et qui permet d'ajouter la valeur **x** dans l'ABR **a**. (ce sera un nouveau noeud placé correctement dans l'arbre).

Affiche_arbre(insérer(C,36)) : affiche **{{{_,24,_},29,{{_,31,_},32,_}},34,{{_,35,{{_,36,_}}},37,{{_,40,_}}}}**.

Bon courage